

altairsim — Burning an EPROM with the SSM PB1

2026-08-22 · 310bf2b

A machine with an **SSM PB1 2708/2716 EPROM programmer** board in it, and SSM's own programmer software from the PB1 manual. Put some bytes in memory, run the burner, and read the burned chip back out as an Intel HEX file on your host.

```
cd examples/pb1
altairsim pb1.toml
```

You land at the `altairsim>` prompt. The machine is a stock Altair — an 8080, 52K of RAM, a 2SIO console, the front panel — with the **PB1** (`pb1`) added: a control port at `10H` and a 4K programming-socket window at `D000`. It is just a board on an ordinary machine; the only thing unusual is that the console is at port `00` and RAM stops at `CFFF`, so the burner software's own addresses (`CPORT = 10H`, socket window `D000`) stay free.

The four programs

These are SSM's routines from the PB1 manual (© Solid State Music 1978), transcribed **verbatim** — source and assembled Intel HEX:

File	Manual	Origin	What it does
PROG2708.ASM / .HEX	§4.2	0100H	burn a 2708 (1K) from RAM at <code>4000</code> into the socket
PROG2716.ASM / .HEX	§4.3	0100H	burn a 2716 (2K) from RAM at <code>4000</code>
ERACHK.ASM / .HEX	§4.4	0140H	check the socket is erased , print <code>P / F</code>
CPYVFY.ASM / .HEX	§4.5	0180H	verify the burn against the RAM source, print <code>P / F</code>

About the monitor at `F021`

Every one of these programs ends with `JMP F021H`, and the two verify programs also `CALL F009H`. Those are **not arbitrary** — PB1 manual §3.3 spells it out: `F021` is the entry address of the **SSM 8080 monitor**, and `F009` is its console-output routine. The manual also says to adapt them to *your* monitor: patch the low/high origin bytes at `011F / 0120` for the exit, and change the `C0` equate for console output.

altairsim does not ship the SSM 8080 monitor yet (its source listing is being sourced and will be added later). Until then:

- The **burners** don't need it — the burn is finished before the final `JMP`. Set a breakpoint at `F021` to catch the "return to monitor", then save the socket yourself.
- The **verify** programs run, but their `P / F` character goes to the (absent) monitor's console-out, so nothing prints yet. When the SSM monitor is added, they will print as SSM intended. In the meantime, verify a burn with `SAVE` + a host `diff`, or by comparing the socket window to the source in memory.

Burn a 2708 and make a hex file

This is the whole point of the board (issue #382): not "prepare a ROM image with `LOAD ... ROM`", but *run the software a 1970s operator ran*, then keep the result.

```
altairsim> FILL 4000-43FF 00      ; (or LOAD your-data.hex) -- the 1K to burn, at 4000
altairsim> LOAD PROG2708.HEX      ; the SSM 2708 burner (manual §4.2)
altairsim> BREAK F021             ; where the burner "returns to the monitor"
altairsim> RUN 100                ; arms the board, burns the socket, hits the breakpoint
altairsim> SAVE eprom.hex D000-D3FF ; the burned 2708, as an Intel HEX file on your host
```

`eprom.hex` is an ordinary Intel HEX file — that is "making a hex file". The socket window is just readable memory once burned, so `SAVE` reads it off the bus and emits the HEX; nothing on the board writes files.

For a **2716**, use `PROG2716.HEX` and save the 2K window `D000-D7FF`. To **copy** an EPROM, mount a source chip in one socket (or the read-only on-board area) and point the burner's source address at it.

The board itself is documented in the [User Manual](#); the hardware reference is `reference/SSM PB1 EPROM Programmer.md`.